

Python: exercises on recursion

This notebook was generated in pseudocode mode.

Exercise 1: Sum of Numbers (Recursive)

Exercise Description

Write a recursive function to calculate the sum of all integers from 1 to n.

Input Example: n = 5

Output Example: 15

The function should use recursion with a base case and recursive case to solve the problem.

Your solution

```
def sum_recursive(n):  
    # step 1: check if n equals 0 (base case)
```

```
# step 2: if true, return 0  
# step 3: otherwise, return n plus the recursive call with n-1
```

Test your solution

```
# Test the function with the provided example  
assert sum_recursive(5) == 15, "sum_recursive(5) should return 15"  
  
# Test with other values  
assert sum_recursive(0) == 0, "sum_recursive(0) should return 0"  
assert sum_recursive(1) == 1, "sum_recursive(1) should return 1"  
assert sum_recursive(3) == 6, "sum_recursive(3) should return 6"
```

Test your solution

```
# Test with larger values  
assert sum_recursive(10) == 55, "sum_recursive(10) should return 55"  
assert sum_recursive(4) == 10, "sum_recursive(4) should return 10"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 2: Factorial (Recursive)

Exercise Description

Calculate the factorial of a given number n using recursion.

Input Example: $n = 4$

Output Example: 24

The factorial of n (written as $n!$) is the product of all positive integers less than or equal to n . For example, $4! = 4 \times 3 \times 2 \times 1 = 24$.

Your solution

```
def factorial(n):  
    # step 1: check if n equals 0 (base case)  
    # step 2: if true, return 1 (0! = 1)  
    # step 3: otherwise, return n multiplied by the recursive call with n-1
```

Test your solution

```
# Test the function with the provided example  
assert factorial(4) == 24, "factorial(4) should return 24"  
  
# Test with other values  
assert factorial(0) == 1, "factorial(0) should return 1"
```

```
assert factorial(1) == 1, "factorial(1) should return 1"
assert factorial(3) == 6, "factorial(3) should return 6"
```

Test your solution

```
# Test with larger values
assert factorial(5) == 120, "factorial(5) should return 120"
assert factorial(2) == 2, "factorial(2) should return 2"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 3: Fibonacci Sequence (Recursive)

Exercise Description

Generate the n-th Fibonacci number using recursion.

Input Example: n = 6

Output Example: 8

The Fibonacci sequence is: 0, 1, 1, 2, 3, 5, 8, 13, ... where each number is the sum of the two preceding ones.

Your solution

```
def fibonacci(n):  
    # step 1: check if n equals 0 (base case 1)  
    # step 2: if true, return 0  
    # step 3: check if n equals 1 (base case 2)  
    # step 4: if true, return 1  
    # step 5: otherwise, return the sum of fibonacci(n-1) and fibonacci(n-2)
```

Test your solution

```
# Test the function with the provided example  
assert fibonacci(6) == 8, "fibonacci(6) should return 8"  
  
# Test with base cases  
assert fibonacci(0) == 0, "fibonacci(0) should return 0"  
assert fibonacci(1) == 1, "fibonacci(1) should return 1"  
assert fibonacci(2) == 1, "fibonacci(2) should return 1"
```

Test your solution

```
# Test with other values  
assert fibonacci(3) == 2, "fibonacci(3) should return 2"  
assert fibonacci(4) == 3, "fibonacci(4) should return 3"
```

```
assert fibonacci(5) == 5, "fibonacci(5) should return 5"
assert fibonacci(7) == 13, "fibonacci(7) should return 13"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 4: Power of a Number (Recursive)

Exercise Description

Write a recursive function to calculate a^b (a raised to the power of b).

Input Example: a = 2, b = 3

Output Example: 8

The function should use recursion to compute the power operation.

Your solution

```
def power(a, b):  
    # step 1: check if b equals 0 (base case)  
    # step 2: if true, return 1 (any number to the power of 0 is 1)  
    # step 3: otherwise, return a multiplied by the recursive call with power(a, b-1)
```

Test your solution

```
# Test the function with the provided example  
assert power(2, 3) == 8, "power(2, 3) should return 8"  
  
# Test with base case  
assert power(5, 0) == 1, "power(5, 0) should return 1"  
assert power(2, 1) == 2, "power(2, 1) should return 2"
```

Test your solution

```
# Test with other values  
assert power(3, 2) == 9, "power(3, 2) should return 9"  
assert power(4, 2) == 16, "power(4, 2) should return 16"  
assert power(2, 4) == 16, "power(2, 4) should return 16"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 5: Reverse a String (Recursive)

Exercise Description

Reverse a given string using recursion.

Input Example: s = "hello"

Output Example: "olleh"

The function should use recursion to reverse the string character by character.

Your solution

```
def reverse_string(s):  
    # step 1: check if the length of s equals 0 (base case)  
    # step 2: if true, return an empty string  
    # step 3: otherwise, return the last character of s plus the recursive call with s[:-1]
```

Test your solution

```
# Test the function with the provided example  
assert reverse_string("hello") == "olleh", "reverse_string('hello') should return 'olleh'"
```



```
# Test with base case
assert reverse_string("") == "", "reverse_string('') should return ''"
assert reverse_string("a") == "a", "reverse_string('a') should return 'a'"
```

Test your solution

```
# Test with other values
assert reverse_string("abc") == "cba", "reverse_string('abc') should return 'cba'"
assert reverse_string("python") == "nohtyp", "reverse_string('python') should return 'nohtyp'"
assert reverse_string("12345") == "54321", "reverse_string('12345') should return '54321'"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 6: Check Palindrome (Recursive)

Exercise Description

Check if a string is a palindrome using recursion.

Input Example: s = "madam"

Output Example: True

A palindrome is a word that reads the same forwards and backwards.

Your solution

```
def is_palindrome(s):  
    # step 1: check if the length of s is less than or equal to 1 (base case)  
    # step 2: if true, return True (single character or empty string is palindrome)  
    # step 3: check if the first character is not equal to the last character  
    # step 4: if true, return False  
    # step 5: otherwise, return the recursive call with s[1:-1] (middle part of string)
```

Test your solution

```
# Test the function with the provided example  
assert is_palindrome("madam") == True, "is_palindrome('madam') should return True"  
  
# Test with base cases  
assert is_palindrome("") == True, "is_palindrome('') should return True"  
assert is_palindrome("a") == True, "is_palindrome('a') should return True"
```

Test your solution

```
# Test with other values  
assert is_palindrome("racecar") == True, "is_palindrome('racecar') should return True"  
assert is_palindrome("hello") == False, "is_palindrome('hello') should return False"
```

```
assert is_palindrome("level") == True, "is_palindrome('level') should return True"
assert is_palindrome("python") == False, "is_palindrome('python') should return False"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 7: Count Digits (Recursive)

Exercise Description

Count the number of digits in a positive integer using recursion.

Input Example: $n = 12345$

Output Example: 5

The function should use recursion to count digits by repeatedly dividing by 10.

Your solution

```
def count_digits(n):  
    # step 1: check if n is less than 10 (base case)  
    # step 2: if true, return 1 (single digit)  
    # step 3: otherwise, return 1 plus the recursive call with n divided by 10 (integer divis
```

Test your solution

```
# Test the function with the provided example  
assert count_digits(12345) == 5, "count_digits(12345) should return 5"  
  
# Test with base case  
assert count_digits(5) == 1, "count_digits(5) should return 1"  
assert count_digits(9) == 1, "count_digits(9) should return 1"
```

Test your solution

```
# Test with other values  
assert count_digits(123) == 3, "count_digits(123) should return 3"  
assert count_digits(1000) == 4, "count_digits(1000) should return 4"  
assert count_digits(99) == 2, "count_digits(99) should return 2"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 8: Greatest Common Divisor (Recursive)

Exercise Description

Find the greatest common divisor (GCD) of two numbers using recursion and the Euclidean algorithm.

Input Example: $a = 48$, $b = 18$

Output Example: 6

The Euclidean algorithm states that $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$ until b becomes 0.

Your solution

```
def gcd(a, b):  
    # step 1: check if b equals 0 (base case)  
    # step 2: if true, return a  
    # step 3: otherwise, return the recursive call with gcd(b, a % b)
```

Test your solution

```
# Test the function with the provided example  
assert gcd(48, 18) == 6, "gcd(48, 18) should return 6"
```

```
# Test with base case
assert gcd(5, 0) == 5, "gcd(5, 0) should return 5"
assert gcd(7, 1) == 1, "gcd(7, 1) should return 1"
```

Test your solution

```
# Test with other values
assert gcd(12, 8) == 4, "gcd(12, 8) should return 4"
assert gcd(15, 25) == 5, "gcd(15, 25) should return 5"
assert gcd(17, 13) == 1, "gcd(17, 13) should return 1"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 9: Binary Representation (Recursive)

Exercise Description

Convert a decimal number to its binary representation using recursion.

Input Example: $n = 10$

Output Example: "1010"

The function should use recursion to convert the number by repeatedly dividing by 2 and collecting remainders.

Your solution

```
def decimal_to_binary(n):  
    # step 1: check if n equals 0 (base case)  
    # step 2: if true, return "0"  
    # step 3: check if n equals 1 (base case)  
    # step 4: if true, return "1"  
    # step 5: otherwise, return the recursive call with  $n//2$  plus the string of  $n\%2$ 
```

Test your solution

```
# Test the function with the provided example  
assert decimal_to_binary(10) == "1010", "decimal_to_binary(10) should return '1010'"  
  
# Test with base cases  
assert decimal_to_binary(0) == "0", "decimal_to_binary(0) should return '0'"  
assert decimal_to_binary(1) == "1", "decimal_to_binary(1) should return '1'"
```

Test your solution

```
# Test with other values  
assert decimal_to_binary(5) == "101", "decimal_to_binary(5) should return '101'"
```

```
assert decimal_to_binary(8) == "1000", "decimal_to_binary(8) should return '1000'"
assert decimal_to_binary(15) == "1111", "decimal_to_binary(15) should return '1111'"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 10: Sum of Digits (Recursive)

Exercise Description

Calculate the sum of all digits in a positive integer using recursion.

Input Example: $n = 123$

Output Example: 6

The function should use recursion to sum digits by repeatedly dividing by 10 and adding remainders.

Your solution


```
def sum_of_digits(n):  
    # step 1: check if n is less than 10 (base case)  
    # step 2: if true, return n (single digit)  
    # step 3: otherwise, return n%10 plus the recursive call with n//10
```

Test your solution

```
# Test the function with the provided example  
assert sum_of_digits(123) == 6, "sum_of_digits(123) should return 6"  
  
# Test with base case  
assert sum_of_digits(5) == 5, "sum_of_digits(5) should return 5"  
assert sum_of_digits(9) == 9, "sum_of_digits(9) should return 9"
```

Test your solution

```
# Test with other values  
assert sum_of_digits(456) == 15, "sum_of_digits(456) should return 15"  
assert sum_of_digits(1000) == 1, "sum_of_digits(1000) should return 1"  
assert sum_of_digits(99) == 18, "sum_of_digits(99) should return 18"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 11: Sum of List Elements (Recursive)

Exercise Description

Calculate the sum of all elements in a list using recursion.

Input Example: `lst = [1, 2, 3, 4, 5]`

Output Example: 15

The function should use recursion to sum elements by processing the first element and recursively summing the rest.

Your solution

```
def sum_list(lst):  
    # step 1: check if the list is empty (base case)  
    # step 2: if true, return 0  
    # step 3: otherwise, return the first element plus the recursive call with the rest of th
```

Test your solution

```
# Test the function with the provided example  
assert sum_list([1, 2, 3, 4, 5]) == 15, "sum_list([1, 2, 3, 4, 5]) should return 15"
```

```
# Test with base case
assert sum_list([]) == 0, "sum_list([]) should return 0"
assert sum_list([5]) == 5, "sum_list([5]) should return 5"
```

Test your solution

```
# Test with other values
assert sum_list([10, 20, 30]) == 60, "sum_list([10, 20, 30]) should return 60"
assert sum_list([1, -1, 2, -2]) == 0, "sum_list([1, -1, 2, -2]) should return 0"
assert sum_list([7, 8, 9]) == 24, "sum_list([7, 8, 9]) should return 24"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 12: Collatz Conjecture Steps (Recursive)

Exercise Description

Count the number of steps to reach 1 in the Collatz sequence using recursion.

Input Example: $n = 3$

Output Example: 7

The Collatz sequence: if n is even, divide by 2; if n is odd, multiply by 3 and add 1. Continue until reaching 1.

Your solution

```
def collatz_steps(n):  
    # step 1: check if n equals 1 (base case)  
    # step 2: if true, return 0 (no more steps needed)  
    # step 3: check if n is even  
    # step 4: if true, return 1 plus the recursive call with n//2  
    # step 5: otherwise (n is odd), return 1 plus the recursive call with 3*n+1
```

Test your solution

```
# Test the function with the provided example  
assert collatz_steps(3) == 7, "collatz_steps(3) should return 7"  
  
# Test with base case  
assert collatz_steps(1) == 0, "collatz_steps(1) should return 0"  
assert collatz_steps(2) == 1, "collatz_steps(2) should return 1"
```

Test your solution

```
# Test with other values  
assert collatz_steps(4) == 2, "collatz_steps(4) should return 2"
```

```
assert collatz_steps(5) == 5, "collatz_steps(5) should return 5"
assert collatz_steps(6) == 8, "collatz_steps(6) should return 8"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 13: Check if List is Sorted (Recursive)

Exercise Description

Check if a list is sorted in ascending order using recursion.

Input Example: `lst = [1, 2, 3, 4, 5]`

Output Example: `True`

The function should use recursion to check if each element is less than or equal to the next element.

Your solution

```
def is_sorted(lst):  
    # step 1: check if the list has less than 2 elements (base case)  
    # step 2: if true, return True (empty or single element is sorted)  
    # step 3: check if the first element is greater than the second element  
    # step 4: if true, return False (not sorted)  
    # step 5: otherwise, return the recursive call with the rest of the list (lst[1:])
```

Test your solution

```
# Test the function with the provided example  
assert is_sorted([1, 2, 3, 4, 5]) == True, "is_sorted([1, 2, 3, 4, 5]) should return True"  
  
# Test with base cases  
assert is_sorted([]) == True, "is_sorted([]) should return True"  
assert is_sorted([5]) == True, "is_sorted([5]) should return True"
```

Test your solution

```
# Test with other values  
assert is_sorted([1, 3, 2, 4]) == False, "is_sorted([1, 3, 2, 4]) should return False"  
assert is_sorted([1, 1, 2, 3]) == True, "is_sorted([1, 1, 2, 3]) should return True"  
assert is_sorted([5, 4, 3, 2, 1]) == False, "is_sorted([5, 4, 3, 2, 1]) should return False"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 14: Find Maximum in List (Recursive)

Exercise Description

Find the maximum element in a list using recursion.

Input Example: `lst = [3, 1, 4, 1, 5, 9, 2]`

Output Example: 9

The function should use recursion to compare the first element with the maximum of the rest of the list.

Your solution

```
def find_max(lst):  
    # step 1: check if the list has only one element (base case)  
    # step 2: if true, return that element  
    # step 3: find the maximum of the rest of the list using recursion  
    # step 4: return the maximum between the first element and the max of the rest
```

Test your solution

```
# Test the function with the provided example
assert find_max([3, 1, 4, 1, 5, 9, 2]) == 9, "find_max([3, 1, 4, 1, 5, 9, 2]) should return 9"

# Test with base case
assert find_max([5]) == 5, "find_max([5]) should return 5"
assert find_max([10]) == 10, "find_max([10]) should return 10"
```

Test your solution

```
# Test with other values
assert find_max([1, 2, 3]) == 3, "find_max([1, 2, 3]) should return 3"
assert find_max([10, 5, 8, 3]) == 10, "find_max([10, 5, 8, 3]) should return 10"
assert find_max([-1, -5, -3]) == -1, "find_max([-1, -5, -3]) should return -1"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 15: Count Zeros in Number (Recursive)

Exercise Description

Count the number of zeros in a positive integer using recursion.

Input Example: $n = 10203$

Output Example: 2

The function should use recursion to count zeros by examining each digit.

Your solution

```
def count_zeros(n):  
    # step 1: check if n is less than 10 (base case)  
    # step 2: if true, return 1 if n equals 0, otherwise return 0  
    # step 3: get the last digit using n % 10  
    # step 4: if the last digit is 0, add 1 to the recursive call with n // 10  
    # step 5: otherwise, just return the recursive call with n // 10
```

Test your solution

```
# Test the function with the provided example  
assert count_zeros(10203) == 2, "count_zeros(10203) should return 2"  
  
# Test with base cases  
assert count_zeros(0) == 1, "count_zeros(0) should return 1"  
assert count_zeros(5) == 0, "count_zeros(5) should return 0"
```

Test your solution

```
# Test with other values
assert count_zeros(1000) == 3, "count_zeros(1000) should return 3"
assert count_zeros(123) == 0, "count_zeros(123) should return 0"
assert count_zeros(50607) == 2, "count_zeros(50607) should return 2"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 16: Flatten Nested List (Recursive)

Exercise Description

Flatten a nested list structure using recursion.

Input Example: nested_list = [1, [2, 3], [4, [5, 6]], 7]

Output Example: [1, 2, 3, 4, 5, 6, 7]

The function should use recursion to handle arbitrarily nested lists and return a flat list.

Your solution

```
def flatten_list(nested_list):  
    # step 1: create an empty result list  
    # step 2: iterate through each element in the nested list  
        # step 3: check if the current element is a list  
            # step 4: if true, recursively flatten it and extend the result  
        # step 5: otherwise, append the element to the result  
    # step 6: return the result list
```

Test your solution

```
# Test the function with the provided example  
assert flatten_list([1, [2, 3], [4, [5, 6]], 7]) == [1, 2, 3, 4, 5, 6, 7], "flatten_list([1,  
  
# Test with simple cases  
assert flatten_list([1, 2, 3]) == [1, 2, 3], "flatten_list([1, 2, 3]) should return [1, 2, 3]  
assert flatten_list([]) == [], "flatten_list([]) should return []"
```

Test your solution

```
# Test with other nested structures  
assert flatten_list([[1, 2], [3, 4]]) == [1, 2, 3, 4], "flatten_list([[1, 2], [3, 4]]) should  
assert flatten_list([1, [2, [3, [4]]]]) == [1, 2, 3, 4], "flatten_list([1, [2, [3, [4]]]]) sh  
assert flatten_list([[[[1]]]], ) == [1], "flatten_list([[[[1]]]]) should return [1]"
```

